

Law ERP — Startup & First-Boot Guide

Last updated: 2026-08-24
App port: 6969 (dev profile)
DB: PostgreSQL (Supabase pooler)

1. What Happens Automatically on First Boot

When the application starts, two `CommandLineRunner` beans execute in order:

DataInitializer (RBAC Foundation)

Step	What	Idempotent?
1	Tenant types — SOLO, LAW_FIRM	✓
2	System firm — SYSTEM (home of super admin)	✓
3	MFA migration — force-enables MFA for existing SUPER_ADMINS	✓
4	System roles — SUPER_ADMIN, FIRM_ADMIN, ADVOCATE, PARALEGAL, CLIENT	✓
5	Modules — 9 modules (CASE_MANAGEMENT, DOCUMENT_MANAGEMENT, etc.)	✓
6	Permissions — ACCESS, VIEW, CREATE, EDIT, DELETE + module extras per module	✓
7	ModulePermission junction — Links modules → permissions	✓
8	Role→Permission matrix — Maps roles to permissions (FULL/READ_ONLY/OWN)	✓
9	Renewal types — 7 system-wide defaults (Trademark, Patent, etc.)	✓
10	SYSTEM firm modules — All 9 modules enabled for the SYSTEM firm	✓

MasterDataSeeder (Nepal Geography)

Step	What
1	Country — Nepal (NP)
2	Provinces — 7 provinces from nepal/provinces.json
3	Districts — 77 districts from nepal/districts.json

Other @PostConstruct / CommandLineRunner Tasks

Bean	What
JwtUtil	HMAC signing key initialization
ScraperServiceImpl	Thread pool for concurrent court scraping
HearingMatchingService	Backfill legacy hearing matches
AppealDeadlineEngine	Seed in-memory appeal deadline rules

2. Manual Steps After First Boot

Step 1: Run the DB Migration

The `permissions.action` CHECK constraint needs updating to include `CREDENTIAL_VIEW` and `CREDENTIAL_REVEAL` .

```
-- Run this SQL against your PostgreSQL database:
ALTER TABLE permissions DROP CONSTRAINT IF EXISTS permissions_action_check;

ALTER TABLE permissions ADD CONSTRAINT permissions_action_check CHECK (
    action IN (
        'VIEW', 'CREATE', 'EDIT', 'DELETE', 'ACCESS',
        'UPLOAD', 'DOWNLOAD', 'SHARE', 'EXPORT', 'SCHEDULE',
        'UPDATE_STATUS', 'ASSIGN', 'APPROVE', 'REJECT', 'REVIEW',
        'ARCHIVE', 'RESTORE', 'PRINT', 'FORWARD',
        'CREDENTIAL_VIEW', 'CREDENTIAL_REVEAL'
    )
);

ALTER TABLE audit_logs DROP CONSTRAINT IF EXISTS audit_logs_entity_type_check;
```

File location: `src/main/resources/db/migration/V2026_08_21__add_project_management_permissions.sql`

Step 2: Register the First Super Admin

The super admin is NOT auto-created. You must register via API:

```
curl -X POST http://localhost:6969/api/v1/super-admin/register \
-H "Content-Type: application/json" \
-d '{
  "data": {
    "fullName": "System Administrator",
    "username": "superadmin",
    "email": "admin@yourfirm.com",
    "mobileNo": "9800000000",
    "password": "YourSecurePassword123!",
    "secretKey": "cdae9adca9e7cb0ab2afb82e2a2a74b7"
  }
}'
```

The `secretKey` comes from `super-admin.registration-secret` in `application-dev.yml` .

Change this secret before production!

Step 3: Login as Super Admin

```
curl -X POST http://localhost:6969/api/v1/super-admin/login \
-H "Content-Type: application/json" \
-d '{
  "data": {
    "username": "superadmin",
    "password": "YourSecurePassword123!"
  }
}'
```

First login will return an MFA setup response with a QR code URI.

Scan the QR code with Google Authenticator, then call `/api/v1/auth/mfa/validate` .

Step 4: Create a Firm

```
curl -X POST http://localhost:6969/api/v1/firms \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <YOUR_JWT>" \
-d '{
  "data": {
    "name": "My Law Firm",
    "lawFirmCode": "MLF",
    "firmType": "LAW_FIRM",
    "email": "info@myfirm.com",
    "phone": "01-5555555",
    "address": "Kathmandu, Nepal",
    "jurisdiction": "Nepal",
    "adminUsername": "firmadmin",
    "adminEmail": "admin@myfirm.com",
    "adminMobileNo": "9811111111",
    "adminPassword": "AdminPass123!",
    "adminFullName": "Firm Admin"
  }
}'
```

What this automatically does:

1. Creates the firm entity
2. Clones all system roles (FIRM_ADMIN, ADVOCATE, PARALEGAL, CLIENT) with their permissions
3. Creates the FIRM_ADMIN user with the cloned FIRM_ADMIN role
4. Enables all 9 modules for the firm
5. Sends a welcome email to the firm admin
6. Creates audit log entries

Step 5: Login as Firm Admin

```
curl -X POST http://localhost:6969/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{
  "data": {
    "username": "firmadmin",
    "password": "AdminPass123!",
    "lawFirmCode": "MLF"
  }
}'
```

3. Architecture Overview

Permission Enforcement (Single Source of Truth)

All authorization flows through `PermissionEvaluator.require(permissionCode)` :

```
Request → SecurityFilterChain (JWT validation)
      → Controller method
      → permissionEvaluator.require("MODULE:ACTION")
        → SUPER_ADMIN? → bypass (always allowed)
```

→ Other user? → load role → load role's permissions → check if code present
→ Missing? → throw ForbiddenException (403)

No more `@PreAuthorize` on controllers — all access control is permission-code based via `PermissionEvaluator` .

Role Code Centralization

All role codes are defined in `RoleCode.java` :

```
public static final String SUPER_ADMIN = "SUPER_ADMIN";
public static final String FIRM_ADMIN = "FIRM_ADMIN";
public static final String ADVOCATE = "ADVOCATE";
public static final String PARALEGAL = "PARALEGAL";
public static final String CLIENT = "CLIENT";
```

Used by: `DataInitializer` , `FirmServiceImpl` , `UserManagementServiceImpl` , `FirmRoleServiceImpl` .

Permission Scope

Scope	Meaning	Who
GLOBAL	Cross-firm access	SUPER_ADMIN permissions only
TENANT	All records in the firm	FIRM_ADMIN, ADVOCATE, PARALEGAL
ASSIGNED	Records assigned to the user	(reserved for future use)
OWN	User's own records	CLIENT (enforced at service/query layer)

Permission Cache

`PermissionEvaluator` caches permissions per user in a `ConcurrentHashMap` . Cache is invalidated when:

- A user's role changes (`EmployeeServiceImpl` , `UserManagementServiceImpl`)
- Role permissions are updated (`FirmRoleServiceImpl`)
- Password is reset (`UserManagementServiceImpl`)
- User is deactivated (`UserManagementServiceImpl`)

4. Module & Permission Matrix

Module	SUPER_ADMIN	FIRM_ADMIN	ADVOCATE	PARALEGAL	CLIENT
CASE_MANAGEMENT	FULL	FULL	FULL	READ_ONLY	OWN
DOCUMENT_MANAGEMENT	FULL	FULL	FULL	READ_ONLY	OWN
CLIENT_MANAGEMENT	FULL	FULL	READ_ONLY	READ_ONLY	—
BILLING	FULL	FULL	READ_ONLY	—	OWN
CALENDAR	FULL	FULL	FULL	READ_ONLY	OWN
EMPLOYEE	FULL	FULL	—	—	—
REPORTS	FULL	FULL	READ_ONLY	—	—
AUDIT	FULL	FULL	—	—	—
PROJECT_MANAGEMENT	FULL	FULL	READ_ONLY	READ_ONLY	OWN

Access levels:

- `FULL` = ACCESS + VIEW + CREATE + EDIT + DELETE + module extras
- `READ_ONLY` = ACCESS + VIEW only
- `OWN` = ACCESS + VIEW (row-level "own records" filtering at service layer)
- `-` = NO_ACCESS (no permissions assigned)

5. FirmModule System

Each firm has a `FirmModule` row per module, controlling whether that module is enabled.

- **New firms** → all modules enabled by default
- **SYSTEM firm** → all modules enabled by default
- **Super Admin** can enable/disable modules per firm via `POST /api/v1/super-admin/firms/{firmId}/modules`
- **Firm users** can view their enabled modules via `GET /api/v1/firm/modules`

Note: `requireModuleAccess()` exists in `PermissionEvaluator` but is not yet called from controllers. When wired up, it will check `FirmModule.isEnabled` before allowing access.

6. Security Checklist (Before Go-Live)

Item	Current (Dev)	Required (Prod)
JWT secret	Hardcoded in <code>application-dev.yml</code>	Move to env var / SystemConfig
AES encryption key	Hardcoded in <code>application.yml</code>	Move to env var
Super admin registration secret	Hardcoded in <code>application-dev.yml</code>	Move to env var, use strong secret
MFA bypass (code 123456)	Present for dev testing	Remove before go-live
DB password	Committed in <code>application-dev.yml</code>	Move to env var
CORS origins	localhost only	Add production domain

7. Troubleshooting

"SUPER_ADMIN role not found"

→ `DataInitializer` hasn't run yet. Start the app once, or check logs for seeding errors.

"System firm not found. Run DataInitializer first."

→ Same as above. The `SYSTEM` firm must exist before registering a super admin.

"Module 'X' is not enabled for your firm"

→ The `FirmModule` for that module is disabled. Super admin must enable it via the FirmModule admin API.

"Missing required permission: X:Y"

→ The user's role doesn't have that permission. Firm admin can update role permissions via `PUT /api/v1/firm/roles/{roleId}/permissions`.

Permission changes not taking effect

→ The permission cache may be stale. Changing role permissions automatically invalidates the cache for affected users. If needed, restart the app.

"HTTP method not supported for this endpoint"

→ You're using the wrong HTTP method (e.g., GET on a POST-only endpoint). Check the API docs at `/swagger-ui.html` .

"Endpoint not found"

→ The path doesn't match any controller mapping. Check Swagger or the controller `@RequestMapping` values.